

Mybatis原理概述之配置类解析源码分析

一、项目初始化

1.1 项目初始化

- 使用Springboot加载mybatis，对User类进行增删改查操作

```
@Data
public class User {
    private Long id;
    private String name;
    private Integer age;
    private String email;
}
```

- mapper文件

```
public interface UserMapper {
    User selectById(@Param("id") Long id);
    List<User> selectAll();
    int insert(User user);
    int update(User user);
    int delete(@Param("id") Long id);
}
```

- 对应的xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="wang.zi.jie.myabtis_study.mapper.UserMapper">

    <select id="selectById" resultType="User">
        SELECT id, name, age, email FROM t_user WHERE id = #{id}
    </select>

    <select id="selectAll" resultType="User">
        SELECT id, name, age, email FROM t_user
    </select>

    <insert id="insert" parameterType="User"
        useGeneratedKeys="true" keyProperty="id">
        INSERT INTO t_user (name, age, email) VALUES (#{name}, #{age}, #
{email})
    </insert>

    <update id="update" parameterType="User">
        UPDATE t_user SET name = #{name}, age = #{age}, email = #{email}
        WHERE id = #{id}
    </update>
```

```

    <delete id="delete">
        DELETE FROM t_user WHERE id = #{id}
    </delete>

</mapper>

```

- 扫描路径

```

@SpringBootApplication
@MapperScan("wang.zi.jie.myabtis_study.mapper")
public class MyabtisStudyApplication {

    public static void main(String[] args) {
        SpringApplication.run(MyabtisStudyApplication.class, args);
    }

}

```

- application配置

```

# MyBatis
mybatis.config-location=classpath:mybatis-config.xml
mybatis.mapper-locations=classpath:mapper/*.xml

```

- mybatis-config配置

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="logImpl" value="STDOUT_LOGGING"/>
        <setting name="mapUnderscoreToCamelCase" value="true"/>
    </settings>
    <typeAliases>
        <package name="wang.zi.jie.myabtis_study.entity"/>
    </typeAliases>
</configuration>

```

1.2 MyBatis的整体执行流程

- 首先看整体流程图

```

[启动阶段]
mybatis-config.xml / application.properties + 注解
    ↓
Configuration
    ↓
注册 MappedStatement

```



- 在启动阶段构建 `SqlSessionFactory`
 - 解析配置
 - 读取 `mybatis-config.xml` 或 Spring Boot 中的 `mybatis.configuration` 属性，得到一个 `Configuration` 对象。
 - 这个对象里包含了：全局设置（如驼峰映射、缓存开关）、类型别名、类型处理器、插件、环境配置（数据源、事务工厂）等。
 - 解析 Mapper
 - 扫描 XML Mapper 文件（`<mapper resource="...">`）或 Mapper 接口（`@Mapper` + `@Select` 等注解）。
 - 每条 SQL 被解析成一个 `MappedStatement` 对象（ID = 命名空间 + 方法名，例如 `com.example.UserMapper.selectById`），并注册到 `Configuration` 中。
 - `MappedStatement` 里面封装了：SQL 语句（可能带 `#{}` 占位符）、参数类型、结果映射规则、是否使用二级缓存等信息。
 - 生成 `SqlSessionFactory`
 - 通常通过 `SqlSessionFactoryBuilder` 读取 `Configuration` 构建 `DefaultSqlSessionFactory` 实例。
 - 在 Spring Boot 中，这个过程由 `SqlSessionFactoryBean` 完成。
- 运行阶段（执行 SQL）、
 - 假设你已经从 Spring 注入了一个 Mapper 接口，并调用了它的一个方法，例如：

```
User user = userMapper.selectById(101);
```

内部流程如下：

1. 获取 `SqlSession`
 - Spring 管理的 `SqlSessionTemplate` 或 MyBatis 原生的 `DefaultSqlSession`，负责持有数据库连接和执行 SQL。
2. Mapper 代理对象

- MyBatis 使用 JDK 动态代理为 Mapper 接口生成代理对象。
- 当你调用 `userMapper.selectById(101)` 时，代理对象会：
 - 根据接口全名 + 方法名，从 `Configuration` 中找到对应的 `MappedStatement`（例如 `com.example.UserMapper.selectById`）。

3. 创建 `BoundSql`（SQL 语句 + 参数）

- `MappedStatement` 中有一个 `SqlSource`，它负责将原始 SQL（带 `#{}` ）解析成最终可执行的 SQL，并提取参数占位符信息。
- 解析结果保存在 `BoundSql` 中，包含：
 - SQL 字符串（`SELECT * FROM user WHERE id = ?`）
 - 参数映射（哪个 Java 属性对应哪个 `?`）
 - 附加参数（如 `_parameter`）

4. 执行器 `Executor`

- MyBatis 中的执行器（`Executor`）负责整个执行流程。常见类型：
 - `SimpleExecutor`：每次执行都创建新的 `PreparedStatement`。
 - `ReuseExecutor`：缓存 `PreparedStatement`。
 - `BatchExecutor`：批量执行。
- 执行器内部会：
 - 检查一级缓存（`SqlSession` 范围内）是否有结果。
 - 如果没有，则调用 `StatementHandler`。

5. `StatementHandler`（数据库交互）

- 负责创建 `PreparedStatement`，并设置参数。
- 具体步骤：
 - a. `ParameterHandler` 将 `BoundSql` 中的参数值设置到 `PreparedStatement` 的 `?` 位置。
 - b. 执行 SQL（`execute()`）。
 - c. 获取 `ResultSet`。

6. `ResultSetHandler`（结果映射）

- 遍历 `ResultSet`，根据 `MappedStatement` 中定义的结果映射规则（`resultMap` 或 `resultType`），将每一行数据转换为 Java 对象。
- 支持简单类型、`Map`、`POJO`、集合、嵌套查询等。
- 最终得到结果对象（如 `User` 或 `List<User>`）。

7. 返回结果

- 结果对象沿着调用链返回：`ResultSetHandler` → `Executor` → `SqlSession` → `Mapper 代理` → 你的业务代码。

二、Mybatis配置类解析源码分析

- 首先需要确定源码从哪里看，Spring Boot 启动时，会根据 `spring.factories` 中的配置加载 `MybatisAutoConfiguration`。它负责：
 - 检查是否有 `SqlSessionFactory`、`SqlSessionTemplate` 等 Bean
 - 读取 `application.properties` 中 `mybatis.*` 开头的属性（通过 `MybatisProperties`）
 - 决定使用哪种方式构建 `Configuration` 对象（纯 properties / configLocation XML / 混合）
- 所以 `application.properties` 和 `mybatis-config.xml` 的自动解析入口在 Spring Boot 中是 `MybatisAutoConfiguration` 这个类。[源码链路从这里分叉](#)
 - 如果项目中 **没有** 指定 `mybatis.config-location`（指向 `mybatis-config.xml`），那么 `MybatisAutoConfiguration` 会直接使用 `MybatisProperties` 中定义的 `Configuration` 对象（通过反射 setter 赋值），**不会调用** `XMLConfigBuilder`。
 - 如果 **有** `mybatis.config-location`，则 `MybatisAutoConfiguration` 会创建 `XMLConfigBuilder` 去解析 XML 文件，构建原生的 `Configuration` 对象，并忽略 `mybatis.configuration.*` 属性。
- 我们为了看mybatis流程，使用mybatis.config-location这个链路分析

2.1 MybatisAutoConfiguration

- 首先看这个类的注解信息

```
@org.springframework.context.annotation.Configuration(proxyBeanMethods =
false)
@ConditionalOnClass({ SqlSessionFactory.class, SqlSessionFactoryBean.class })
@ConditionalOnSingleCandidate(DataSource.class)
@EnableConfigurationProperties(MybatisProperties.class)
@AutoConfigureAfter({ DataSourceAutoConfiguration.class,
MybatisLanguageDriverAutoConfiguration.class })
public class MybatisAutoConfiguration implements InitializingBean {
    //省略
}
```

- `@Configuration(proxyBeanMethods = false)`
 - **作用**：标明这是一个 Spring 配置类，里面通常会定义 `@Bean` 方法。
 - `proxyBeanMethods = false`：告诉 Spring 不要为这个配置类生成 CGLIB 代理（不会拦截 `@Bean` 方法的内部调用）。好处是启动更快、更轻量，缺点是不能通过方法调用共享 Bean 实例（即必须依赖参数注入）。在 MyBatis 的自动配置中，没有这种内部方法调用的需求，所以用 `false` 提升性能。
- `@ConditionalOnClass({ SqlSessionFactory.class, SqlSessionFactoryBean.class })`
 - **条件注解**：只有当类路径下**同时存在** `SqlSessionFactory` 和 `SqlSessionFactoryBean` 这两个类时，当前配置类才会生效。
 - **意义**：确保项目里引入了 `mybatis-spring` 或 MyBatis 的核心依赖，否则自动配置不会干扰 Spring Boot 的正常启动
- `@ConditionalOnSingleCandidate(DataSource.class)`

- **条件注解**: 要求 Spring 容器中有且仅有一个 `DataSource` 类型的候选 Bean (或者虽然有多个, 但有一个被标记为 `@Primary`)。
- **意义**: MyBatis 需要访问数据库, 所以必须依赖一个唯一的 `DataSource`。如果用户没有配置数据源, 或者存在多个且未指定 `@Primary`, 这个自动配置会跳过, 避免歧义。
- `@EnableConfigurationProperties(MybatisProperties.class)`
 - **作用**: 将 `MybatisProperties` 这个带 `@ConfigurationProperties(prefix = "mybatis")` 的类注册为 Spring Bean, 并自动绑定 `application.properties` 或 `application.yml` 中以 `mybatis` 开头的配置项。
 - **效果**: 例如配置 `mybatis.config-location`、`mybatis.type-aliases-package` 等, 都会被映射到 `MybatisProperties` 对象的对应字段中。
 - 当 `@EnableConfigurationProperties(MybatisProperties.class)` 被使用时, Spring Boot 在启动阶段会做两件事:
 1. 自动调用 `MybatisProperties` 的无参构造器, 创建一个实例。
 2. 通过 `ConfigurationPropertiesBindingPostProcessor` (一个 Bean 后置处理器), 将 `application.properties` 中以 `mybatis` 为前缀的配置值绑定到该实例的字段上。

```
@ConfigurationProperties(prefix = MybatisProperties.MYBATIS_PREFIX)
public class MybatisProperties {

    public static final String MYBATIS_PREFIX = "mybatis";

    private static final ResourcePatternResolver resourceResolver =
new PathMatchingResourcePatternResolver();

    /**
     * Location of MyBatis xml config file.
     */
    private String configLocation;

    /**
     * Locations of MyBatis mapper files.
     */
    private String[] mapperLocations;

    /**
     * Packages to search type aliases. (Package delimiters are ",;
\t\n")
     */
    private String typeAliasesPackage;

    /**
     * The super class for filtering type alias. If this not
specifies, the MyBatis deal as type alias all classes that
     * searched from typeAliasesPackage.
     */
    private Class<?> typeAliasesSuperType;
```

- `@AutoConfigureAfter({ DataSourceAutoConfiguration.class, MybatisLanguageDriverAutoConfiguration.class })`
 - **作用**：指定本配置类需要在 `DataSourceAutoConfiguration`（数据源自动配置）和 `MybatisLanguageDriverAutoConfiguration`（MyBatis 语言驱动自动配置）**之后**执行。
 - **意义**：保证 `DataSource` 已经先被配置好，以及 MyBatis 的语言驱动（用于动态 SQL 等）就绪后，再创建 `SqlSessionFactory`。避免出现依赖未准备好的问题。
- `implements InitializingBean`
 - **接口**：Spring 会在所有属性设置完成后（包括 `MybatisProperties` 注入）调用 `afterPropertiesSet()` 方法。
 - **在该类中的应用**：通常会在这个方法中对 `MybatisProperties` 做一些**校验**（比如检查 `configLocation` 配置的 XML 文件是否存在），或者设置一些**默认值**，确保后续构建 `SqlSessionFactory` 时配置是完整合法的。
- 剩余先不看，**追源码不是从头读到尾，而是用已知的关键点（类名、方法名、注解）做“跳转”，一步步逼近你要理解的那一小块逻辑。**
 - 我们从总流程看到了 `SqlSessionFactory`，所以我们先看这个方法

```
@Bean
@ConditionalOnMissingBean
public SqlSessionFactory sqlSessionFactory(DataSource dataSource)
throws Exception {
    SqlSessionFactoryBean factory = new SqlSessionFactoryBean();
    factory.setDataSource(dataSource);
    if (properties.getConfiguration() == null ||
properties.getConfiguration().getVfsImpl() == null) {
        factory.setVfs(SpringBootVFS.class);
    }
    if (StringUtils.hasText(this.properties.getConfigLocation())) {
        factory.setConfigLocation(this.resourceLoader.getResource(this.properties.getConfigLocation()));
    }
    applyConfiguration(factory);
    if (this.properties.getConfigurationProperties() != null) {
        factory.setConfigurationProperties(this.properties.getConfigurationProperties());
    }
    if (!ObjectUtils.isEmpty(this.interceptors)) {
        factory.setPlugins(this.interceptors);
    }
    if (this.databaseIdProvider != null) {
        factory.setDatabaseIdProvider(this.databaseIdProvider);
    }
    if (StringUtils.hasLength(this.properties.getTypeAliasesPackage())) {
        factory.setTypeAliasesPackage(this.properties.getTypeAliasesPackage());
    }
    if (this.properties.getTypeAliasesSuperType() != null) {

```

```

        factory.setTypeAliasesSuperType(this.properties.getTypeAliasesSuperType(
));
    }
    if (StringUtils.hasLength(this.properties.getTypeHandlersPackage()))
    {

        factory.setTypeHandlersPackage(this.properties.getTypeHandlersPackage())
        ;
    }
    if (!ObjectUtils.isEmpty(this.typeHandlers)) {
        factory.setTypeHandlers(this.typeHandlers);
    }
    Resource[] mapperLocations =
this.properties.resolveMapperLocations();
    if (!ObjectUtils.isEmpty(mapperLocations)) {
        factory.setMapperLocations(mapperLocations);
    }
    Set<String> factoryPropertyNames = Stream
        .of(new
BeanWrapperImpl(SqlSessionFactoryBean.class).getPropertyDescriptors()).ma
p(PropertyDescriptor::getName)
        .collect(Collectors.toSet());
    Class<? extends LanguageDriver> defaultLanguageDriver =
this.properties.getDefaultScriptingLanguageDriver();
    if (factoryPropertyNames.contains("scriptingLanguageDrivers") &&
!ObjectUtils.isEmpty(this.languageDrivers)) {
        // Need to mybatis-spring 2.0.2+
        factory.setScriptingLanguageDrivers(this.languageDrivers);
        if (defaultLanguageDriver == null && this.languageDrivers.length ==
1) {
            defaultLanguageDriver = this.languageDrivers[0].getClass();
        }
    }
    if (factoryPropertyNames.contains("defaultScriptingLanguageDriver"))
    {
        // Need to mybatis-spring 2.0.2+
        factory.setDefaultScriptingLanguageDriver(defaultLanguageDriver);
    }
    applySqlSessionFactoryBeanCustomizers(factory);
    return factory.getObject();
}

```

- 先看方法体
 - `@Bean`：声明该方法返回的对象会注册为 Spring 容器中的 Bean。
 - `@ConditionalOnMissingBean`：只有当容器中还没有 `SqlSessionFactory` 类型的 Bean 时，才会执行这个方法。这样用户可以手动定义自己的 `SqlSessionFactory` 来覆盖自动配置。
 - 参数 `DataSource dataSource`：Spring 会自动注入当前容器中唯一（或有 `@Primary`）的数据源。
- 创建 `SqlSessionFactoryBean` 并设置基本属性


```
SqlSessionFactoryBean factory = new SqlSessionFactoryBean();
factory.setDataSource(dataSource);
```

- `SqlSessionFactoryBean` 是 Spring 对 MyBatis 原生 `SqlSessionFactoryBuilder` 的封装，它实现了 `FactoryBean<SqlSessionFactory>`，便于在 Spring 环境中构建 `SqlSessionFactory`。
- 先设置数据源 —— 这是 MyBatis 连接数据库的基础。
- 设置 VFS（虚拟文件系统）

```
if (properties.getConfiguration() == null ||
    properties.getConfiguration().getVfsImpl() == null) {
    factory.setVfs(SpringBootVFS.class);
}
```

- MyBatis 需要扫描类路径下的资源（如 Mapper XML 文件）。VFS 负责这个扫描过程。
 - 如果用户没有在 `mybatis.configuration.vfsImpl` 中指定自定义 VFS 实现，则默认使用 `SpringBootVFS`（Spring Boot 提供的优化版本，能正确识别嵌套 JAR 中的资源）。
- 处理 `mybatis-config.xml` 配置

```
if (StringUtils.hasText(this.properties.getConfigLocation())) {

    factory.setConfigLocation(this.resourceLoader.getResource(this.properties.getConfigLocation()));
}
```

- 如果用户在 `application.properties` 中指定了 `mybatis.config-location`（比如 `classpath:mybatis-config.xml`），则把该 XML 文件的位置设置到 `factory` 中。
 - **关键点：**一旦设置了 `configLocation`，`SqlSessionFactoryBean` 会优先使用 `XMLConfigBuilder` 解析该 XML 文件来构建 `Configuration` 对象，而忽略 `mybatis.configuration.*` 的属性配置。
- 应用 `applyConfiguration(factory)` —— 核心分叉点

```
applyConfiguration(factory);
//展开下
private void applyConfiguration(SqlSessionFactoryBean factory) {
    Configuration configuration = this.properties.getConfiguration();
    if (configuration == null &&
        !StringUtils.hasText(this.properties.getConfigLocation())) {
        //这里就是spring自动补齐，保证后续
        factory.setConfiguration(configuration) 有对象可用，避免空指针或完全无配置的情况。

        configuration = new Configuration();
    }
    if (configuration != null &&
        !StringUtils.hasText(this.properties.getConfigLocation())) {
        factory.setConfiguration(configuration);
    }
}
```

- `this.properties.getConfiguration()` 返回的是通过 `@ConfigurationProperties` 绑定的 `mybatis.configuration.*` 属性 (如 `map-underscore-to-camel-case`、`cache-enabled` 等), 它们会被自动封装成一个 `Configuration` 对象。

- **逻辑:**

- 如果用户**没有指定** `configLocation` 且 `properties.getConfiguration()` 不为空 (或者为空但自动创建新实例), 则把这个 `Configuration` 对象直接设置到 `factory` 中——这就是“**Spring Boot 帮你拼 Configuration 对象**”的方式。
- 如果用户**指定了** `configLocation`, 则不会调用 `factory.setConfiguration(...)`, 而是留待 `factory` 内部通过解析 XML 来生成 `Configuration`。

- 设置全局配置属性

```
if (this.properties.getConfigurationProperties() != null) {  
  
    factory.setConfigurationProperties(this.properties.getConfigurationProperties());  
}
```

- `mybatis.configuration-properties` 允许你设置一组键值对 (`<properties>` 标签的内容), 可以在 SQL 映射文件中通过 `${key}` 引用。这里将整个 `Properties` 对象传递给 `factory`。

- 注册插件

```
if (!ObjectUtils.isEmpty(this.interceptors)) {  
    factory.setPlugins(this.interceptors);  
}
```

- `this.interceptors` 是从 Spring 容器中自动收集的所有 `Interceptor` 类型的 Bean (通过 `@Autowired(required=false)` 注入)。
- MyBatis 插件可以拦截 `Executor`、`StatementHandler`、`ParameterHandler`、`ResultSetHandler` 的行为。这里把所有找到的插件设置到 `factory` 中, 最终会注册到 MyBatis 的 `Configuration` 里。

- 设置 `DatabaseIdProvider`

```
if (this.databaseIdProvider != null) {  
    factory.setDatabaseIdProvider(this.databaseIdProvider);  
}
```

- `DatabaseIdProvider` 用于根据当前数据库产品 (如 MySQL、Oracle) 提供 `databaseId`, 从而允许在 Mapper 中根据不同数据库写不同的 SQL (`<select databaseId="mysql">`)。
- 如果 Spring 容器中有 `DatabaseIdProvider` 的 Bean, 会自动注入并使用。

- 类型别名 (`TypeAliases`)

```

if (StringUtils.hasLength(this.properties.getTypeAliasesPackage())) {

    factory.setTypeAliasesPackage(this.properties.getTypeAliasesPackage());
}
if (this.properties.getTypeAliasesSuperType() != null) {

    factory.setTypeAliasesSuperType(this.properties.getTypeAliasesSuperType(
));
}

```

- `mybatis.type-aliases-package`: 指定扫描类型别名的包, MyBatis 会将该包下的类注册为小写非限定类名 (如 `com.example.User` → `user`) 。
 - `mybatis.type-aliases-super-type`: 如果指定了父类, 则只会扫描该父类的子类作为别名类。
- 类型处理器 (TypeHandlers)

```

if (StringUtils.hasLength(this.properties.getTypeHandlersPackage())) {

    factory.setTypeHandlersPackage(this.properties.getTypeHandlersPackage())
;
}
if (!ObjectUtils.isEmpty(this.typeHandlers)) {
    factory.setTypeHandlers(this.typeHandlers);
}

```

- `typeHandlersPackage`: 扫描指定包下的所有类型处理器, 自动注册。
 - `this.typeHandlers`: 从 Spring 容器中收集所有 `TypeHandler` 类型的 Bean, 手动注册。
- 设置 Mapper XML 文件位置

```

Resource[] mapperLocations = this.properties.resolveMapperLocations();
if (!ObjectUtils.isEmpty(mapperLocations)) {
    factory.setMapperLocations(mapperLocations);
}

```

- `mybatis.mapper-locations`: 默认值为 `classpath*:mapper/**/*.xml`。该方法将这些路径解析为 `Resource[]` 数组, 告诉 `SqlSessionFactoryBean` 到这些位置加载 Mapper XML 文件。
 - 每个 XML 文件会被 `XMLMapperBuilder` 解析, 生成 `MappedStatement` 并注册到 `configuration` 中。
- 脚本语言驱动 (LanguageDriver) 支持

```

Set<String> factoryPropertyNames = ... // 反射获取 SqlSessionFactoryBean
所有属性名
Class<? extends LanguageDriver> defaultLanguageDriver =
this.properties.getDefaultScriptingLanguageDriver();

if (factoryPropertyNames.contains("scriptingLanguageDrivers") &&
!ObjectUtils.isEmpty(this.languageDrivers)) {
    factory.setScriptingLanguageDrivers(this.languageDrivers);
    if (defaultLanguageDriver == null && this.languageDrivers.length ==
1) {
        defaultLanguageDriver = this.languageDrivers[0].getClass();
    }
}
if (factoryPropertyNames.contains("defaultScriptingLanguageDriver")) {
    factory.setDefaultScriptingLanguageDriver(defaultLanguageDriver);
}

```

- 这部分是为了兼容 mybatis-spring 2.0.2+ 版本中新增的属性。
- languageDrivers 是从 Spring 容器中收集的 LanguageDriver Bean（例如 XMLLanguageDriver、RawLanguageDriver）。
- 可以设置默认的语言驱动，用于执行动态 SQL（如 <if>、<foreach>）。
- 应用自定义器（Customizers）

```

applySqlSessionFactoryBeanCustomizers(factory);

```

- 该内部方法会遍历所有 SqlSessionFactoryBeanCustomizer 类型的 Bean，允许用户在自动配置的基础上对 SqlSessionFactoryBean 进行额外定制（比如修改某些属性）。这是 Spring Boot 提供的扩展点。
- 返回 SqlSessionFactory 实例

```

return factory.getObject();

```

- factory.getObject() 内部会调用
SqlSessionFactoryBean.afterPropertiesSet() →
buildSqlSessionFactory(), 真正构建 SqlSessionFactory 并返回。
- 正是这一步最终触发了 MyBatis 的启动流程（解析 XML/注解、注册 MappedStatement 等）。

2.2 配置文件

- 打开上面说的factory.getObject()

```

public SqlSessionFactory getObject() throws Exception {
    if (this.sqlSessionFactory == null) {
        this.afterPropertiesSet();
    }

    return this.sqlSessionFactory;
}

```

- `this.sqlSessionFactory`: 是 `SqlSessionFactoryBean` 内部缓存的一个字段, 真正要返回的对象。
- 第一次调用时, 它为 `null`, 于是调用 `afterPropertiesSet()` (Spring 的 `InitializingBean` 接口方法)。
- `afterPropertiesSet()` 内部会执行真正的构建逻辑:
 - 解析 `mybatis-config.xml` 或 `application.properties` 中的配置;
 - 创建 `Configuration` 对象;
 - 扫描 Mapper 接口和 XML 文件;
 - 最终生成一个 `DefaultSqlSessionFactory` 实例, 并赋值给 `this.sqlSessionFactory`。
- 之后再次调用 `getObject()`, 直接返回已创建好的实例, 避免重复构建。
- 点击 `this.afterPropertiesSet()`;

```
public void afterPropertiesSet() throws Exception {
    Assert.notNull(this.dataSource, "Property 'dataSource' is required");
    Assert.notNull(this.sqlSessionFactoryBuilder, "Property
'sqlSessionFactoryBuilder' is required");
    Assert.state(this.configuration == null && this.configLocation ==
null || this.configuration == null || this.configLocation == null, "Property
'configuration' and 'configLocation' can not specified with together");
    this.sqlSessionFactory = this.buildSqlSessionFactory();
}
```

- 确保 `dataSource` 和 `sqlSessionFactoryBuilder` 这两个必要的属性已经被注入 (通常是 Spring 容器自动注入的)。
- 如果任何一个为空, Spring 启动时会抛出 `IllegalArgumentException`, 明确告知缺少必要配置。
- 这个断言检查了一个重要规则: `configuration` 对象和 `configLocation` (指向 `mybatis-config.xml` 的路径) 不能同时存在。

逻辑拆解:

- `configuration` 对应 `application.properties` 中 `mybatis.configuration.*` 自动绑定的 `Configuration` 对象。
- `configLocation` 对应 `mybatis.config-location` 指向的 XML 文件路径。
- 允许的情况:
 - 两者都为空 → 后续会自动创建一个新的 `Configuration` 对象。
 - 仅 `configuration` 不为空 → 使用用户编程方式提供的 `Configuration` 对象 (对应 `application.properties` 中的 `mybatis.configuration.*` 方式)。
 - 仅 `configLocation` 不为空 → 使用 XML 文件方式解析构建 `Configuration` 对象。
- 禁止的情况:
 - 两者都不为空 → 抛出 `IllegalStateException`, 因为 MyBatis 无法决定以哪个为准。
- 不属于 `configuration` 对象的配置仍然可以正常使用:

- 调用 `buildSqlSessionFactory()` 方法，该方法内部会根据上一步的校验结果，选择路径：
 - 如果有 `configLocation` → 使用 `XMLConfigBuilder` 解析 XML。
 - 否则，如果有 `configuration` 对象 → 直接使用该对象。
 - 否则，创建一个新的 `Configuration` 对象。
- 最终生成 `DefaultSqlSessionFactory` 实例，并赋值给成员变量 `sqlSessionFactory`，供 `getObject()` 返回。
- 点击protected SqlSessionFactory buildSqlSessionFactory() throws Exception 方法

```
protected SqlSessionFactory buildSqlSessionFactory() throws Exception {

    final Configuration targetConfiguration;

    XMLConfigBuilder xmlConfigBuilder = null;
    if (this.configuration != null) {
        targetConfiguration = this.configuration;
        if (targetConfiguration.getVariables() == null) {
            targetConfiguration.setVariables(this.configurationProperties);
        } else if (this.configurationProperties != null) {
            targetConfiguration.getVariables().putAll(this.configurationProperties);
        }
    } else if (this.configLocation != null) {
        xmlConfigBuilder = new
XMLConfigBuilder(this.configLocation.getInputStream(), null,
this.configurationProperties);
        targetConfiguration = xmlConfigBuilder.getConfiguration();
    } else {
        LOGGER.debug(
            () -> "Property 'configuration' or 'configLocation' not specified,
using default MyBatis Configuration");
        targetConfiguration = new Configuration();

        Optional.ofNullable(this.configurationProperties).ifPresent(targetConfigurat
ion::setVariables);
    }

    Optional.ofNullable(this.objectFactory).ifPresent(targetConfiguration::setOb
jectFactory);

    Optional.ofNullable(this.objectWrapperFactory).ifPresent(targetConfiguration
::setObjectWrapperFactory);
    Optional.ofNullable(this.vfs).ifPresent(targetConfiguration::setVfsImpl);

    if (hasLength(this.typeAliasesPackage)) {
        scanClasses(this.typeAliasesPackage,
this.typeAliasesSuperType).stream()
            .filter(clazz -> !clazz.isAnonymousClass()).filter(clazz ->
!clazz.isInterface())
            .filter(clazz ->
!clazz.isMemberClass()).forEach(targetConfiguration.getTypeAliasRegistry::r
egisterAlias);
    }
}
```

```

    if (!isEmpty(this.typeAliases)) {
        Stream.of(this.typeAliases).forEach(typeAlias -> {
            targetConfiguration.getTypeAliasRegistry().registerAlias(typeAlias);
            LOGGER.debug(() -> "Registered type alias: '" + typeAlias + "'");
        });
    }

    if (!isEmpty(this.plugins)) {
        Stream.of(this.plugins).forEach(plugin -> {
            targetConfiguration.addInterceptor(plugin);
            LOGGER.debug(() -> "Registered plugin: '" + plugin + "'");
        });
    }

    if (hasLength(this.typeHandlersPackage)) {
        scanClasses(this.typeHandlersPackage,
            TypeHandler.class).stream().filter(clazz -> !clazz.isAnonymousClass())
            .filter(clazz -> !clazz.isInterface()).filter(clazz ->
                !Modifier.isAbstract(clazz.getModifiers()))
            .forEach(targetConfiguration.getTypeHandlerRegistry()::register);
    }

    if (!isEmpty(this.typeHandlers)) {
        Stream.of(this.typeHandlers).forEach(typeHandler -> {
            targetConfiguration.getTypeHandlerRegistry().register(typeHandler);
            LOGGER.debug(() -> "Registered type handler: '" + typeHandler + "'");
        });
    }

    targetConfiguration.setDefaultEnumTypeHandler(defaultEnumTypeHandler);

    if (!isEmpty(this.scriptingLanguageDrivers)) {
        Stream.of(this.scriptingLanguageDrivers).forEach(languageDriver -> {
            targetConfiguration.getLanguageRegistry().register(languageDriver);
            LOGGER.debug(() -> "Registered scripting language driver: '" +
                languageDriver + "'");
        });
    }

    Optional.ofNullable(this.defaultScriptingLanguageDriver)
        .ifPresent(targetConfiguration::setDefaultScriptingLanguage);

    if (this.databaseIdProvider != null) {// fix #64 set databaseId before
parse mapper xmls
        try {

            targetConfiguration.setDatabaseId(this.databaseIdProvider.getDatabaseId(this
                .dataSource));
        } catch (SQLException e) {
            throw new IOException("Failed getting a databaseId", e);
        }
    }

    Optional.ofNullable(this.cache).ifPresent(targetConfiguration::addCache);

    if (xmlConfigBuilder != null) {

```

```

        try {
            xmlConfigBuilder.parse();
            LOGGER.debug(() -> "Parsed configuration file: '" +
this.configLocation + "'");
        } catch (Exception ex) {
            throw new IOException("Failed to parse config resource: " +
this.configLocation, ex);
        } finally {
            ErrorContext.instance().reset();
        }
    }

    targetConfiguration.setEnvironment(new Environment(this.environment,
        this.transactionFactory == null ? new
SpringManagedTransactionFactory() : this.transactionFactory,
        this.dataSource));

    if (this.mapperLocations != null) {
        if (this.mapperLocations.length == 0) {
            LOGGER.warn(() -> "Property 'mapperLocations' was specified but
matching resources are not found.");
        } else {
            for (Resource mapperLocation : this.mapperLocations) {
                if (mapperLocation == null) {
                    continue;
                }
                try {
                    var xmlMapperBuilder = new
XMLMapperBuilder(mapperLocation.getInputStream(), targetConfiguration,
                        mapperLocation.toString(),
targetConfiguration.getSqlFragments());
                    xmlMapperBuilder.parse();
                } catch (Exception e) {
                    throw new IOException("Failed to parse mapping resource: '" +
mapperLocation + "'", e);
                } finally {
                    ErrorContext.instance().reset();
                }
                LOGGER.debug(() -> "Parsed mapper file: '" + mapperLocation + "'");
            }
        }
    } else {
        LOGGER.debug(() -> "Property 'mapperLocations' was not specified.");
    }

    return this.sqlSessionFactoryBuilder.build(targetConfiguration);
}

```

- Configuration对象的来源，这个就是上面讲的三种情况，最后一种就是互斥

```

final Configuration targetConfiguration;

XMLConfigBuilder xmlConfigBuilder = null;
if (this.configuration != null) {
    targetConfiguration = this.configuration;
}

```



```

        if (targetConfiguration.getVariables() == null) {
            targetConfiguration.setVariables(this.configurationProperties);
        } else if (this.configurationProperties != null) {

targetConfiguration.getVariables().putAll(this.configurationProperties);
        }
    } else if (this.configLocation != null) {
        xmlConfigBuilder = new
XMLConfigBuilder(this.configLocation.getInputStream(), null,
this.configurationProperties);
        targetConfiguration = xmlConfigBuilder.getConfiguration();
    } else {
        LOGGER.debug(
            () -> "Property 'configuration' or 'configLocation' not
specified, using default MyBatis Configuration");
        targetConfiguration = new Configuration();

        Optional.ofNullable(this.configurationProperties).ifPresent(targetConfig
uration::setVariables);
    }

```

分支	条件	结果	对应方式
分支 1	<code>this.configuration != null</code>	直接使用该对象（由 <code>application.properties</code> 的 <code>mybatis.configuration.*</code> 绑定而来）	Spring Boot 帮你拼 Configuration
分支 2	<code>this.configLocation != null</code>	创建 <code>XMLConfigBuilder</code> ，但此时尚未调用 <code>parse()</code> ，只先拿到空的 <code>Configuration</code>	MyBatis 原生 XML 解析准备
分支 3	两者都 <code>null</code>	创建一个全新的 <code>Configuration</code> 对象（使用默认值）	纯粹的默认配置

- **注意：**分支2中虽然创建了 `xmlConfigBuilder` 并拿到了 `targetConfiguration`，但此时 XML 文件还没有被解析（`parse()` 在后面调用）。这是为了保证先注册 Spring 环境中的组件（如插件、类型别名等），然后 XML 中的配置可以覆盖或补充。
- 下面的代码就是对象工厂、包装工厂、VFS、类型别名（包扫描 + 个别注册）、插件、类型处理器（包扫描 + 个别注册）、枚举类型处理器、脚本语言驱动、数据库标识、缓存

```

// 对象工厂、包装工厂、VFS
Optional.ofNullable(this.objectFactory).ifPresent(targetConfiguration::setObjectFactory);
Optional.ofNullable(this.objectWrapperFactory).ifPresent(targetConfiguration::setObjectWrapperFactory);
Optional.ofNullable(this.vfs).ifPresent(targetConfiguration::setVfsImpl);

// 类型别名（包扫描 + 个别注册）
if (hasLength(this.typeAliasesPackage)) { ... }
if (!isEmpty(this.typeAliases)) { ... }

```

```
// 插件
if (!isEmpty(this.plugins)) { ... }

// 类型处理器（包扫描 + 个别注册）
if (hasLength(this.typeHandlersPackage)) { ... }
if (!isEmpty(this.typeHandlers)) { ... }

// 枚举类型处理器
targetConfiguration.setDefaultEnumTypeHandler(defaultEnumTypeHandler);

// 脚本语言驱动
if (!isEmpty(this.scriptingLanguageDrivers)) { ... }
Optional.ofNullable(this.defaultScriptingLanguageDriver).ifPresent(target
Configuration::setDefaultScriptingLanguage);

// 数据库标识
if (this.databaseIdProvider != null) { ... }

// 缓存
Optional.ofNullable(this.cache).ifPresent(targetConfiguration::addCache);
```

- 解析 mybatis-config.xml，这里就是真正的xml解析所在的地方

```
if (xmlConfigBuilder != null) {
    try {
        xmlConfigBuilder.parse();
        LOGGER.debug(() -> "Parsed configuration file: '" +
this.configLocation + "'");
    } catch (Exception ex) {
        throw new IOException("Failed to parse config resource: " +
this.configLocation, ex);
    } finally {
        ErrorContext.instance().reset();
    }
}
```

- 点击parse方法

```
public Configuration parse() {
    if (this.parsed) {
        throw new BuilderException("Each XMLConfigBuilder can
only be used once.");
    } else {
        this.parsed = true;

        this.parseConfiguration(this.parser.evalNode("/configuration"));
        return this.configuration;
    }
}
```

- 我们通过源码可以看到他是通过读configuration标签

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration PUBLIC "-//mybatis.org//DTD Config
3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>
    <settings>
        <setting name="logImpl" value="STDOUT_LOGGING"/>
        <setting name="mapUnderscoreToCamelCase" value="true"/>
    </settings>
    <typeAliases>
        <package name="wang.zi.jie.myabtis_study.entity"/>
    </typeAliases>
</configuration>
```

- 我们可以看到configuration标签的内部标签被解析,每个子元素都有对应的 `xxxElement` 方法,负责具体的解析和注册逻辑,他使用其底层依赖的是 **JDK 自带的 XML 解析 API**,并对它进行了一套精巧的包装,而不是使用 Dom4j 这类流行的第三方库

```
private void parseConfiguration(XNode root) {
    try {
        this.propertiesElement(root.evalNode("properties"));
        Properties settings =
this.settingsAsProperties(root.evalNode("settings"));
        this.loadCustomVfsImpl(settings);
        this.loadCustomLogImpl(settings);

        this.typeAliasesElement(root.evalNode("typeAliases"));
        this.pluginsElement(root.evalNode("plugins"));

        this.objectFactoryElement(root.evalNode("objectFactory"));

        this.objectWrapperFactoryElement(root.evalNode("objectWrapperFactory"));

        this.reflectorFactoryElement(root.evalNode("reflectorFactory"));
        ;
        this.settingsElement(settings);

        this.environmentsElement(root.evalNode("environments"));

        this.databaseIdProviderElement(root.evalNode("databaseIdProvider"));

        this.typeHandlersElement(root.evalNode("typeHandlers"));
        this.mappersElement(root.evalNode("mappers"));
    } catch (Exception var3) {
        Exception e = var3;
        throw new BuilderException("Error parsing SQL Mapper
Configuration. Cause: " + e, e);
    }
}
```

- `propertiesElement(root.evalNode("properties"))`
 - 解析 标签，用于加载外部属性文件或直接定义键值对。这些属性可以在 XML 的其他地方通过 `${key}` 引用，例如 中的 URL、用户名密码等。
- `Properties settings = this.settingsAsProperties(root.evalNode("settings"))`
读取 标签下的所有，转换为 Properties 对象。
 - `settings` 后面会被再次使用（`settingsElement(settings)` 真正应用到 Configuration）。
- `this.loadCustomVfsImpl(settings)`
 - 根据 加载用户自定义的 VFS 实现类（你之前问过 VFS 是什么）
- `this.loadCustomLogImpl(settings)`
 - 根据 设置 MyBatis 使用的日志实现（如 `STDOUT_LOGGING`、`LOG4J` 等）。*
- `this.typeAliasesElement(root.evalNode("typeAliases"))`
 - 解析，注册类型别名。支持：
(批量扫描)
- `this.pluginsElement(root.evalNode("plugins"))`
 - 解析，注册 MyBatis 插件（拦截器）。每个 会通过反射实例化并添加到 Configuration 中。
- `this.objectFactoryElement(root.evalNode("objectFactory"))`
 - 解析，用于自定义 MyBatis 创建结果对象的方式（很少使用）。
- `this.objectWrapperFactoryElement(root.evalNode("objectWrapperFactory"))`
 - 解析，自定义对象包装器（用于扩展对象的属性访问，也很少使用）。
- `this.reflectorFactoryElement(root.evalNode("reflectorFactory"))`
 - 解析，自定义反射工厂（用于缓存类元数据，极少自定义）。
- `this.settingsElement(settings)`
 - 这一步真正将之前收集的 属性应用到 Configuration 对象。例如：
`mapUnderscoreToCamelCase` →
`configuration.setMapUnderscoreToCamelCase(true)` `cacheEnabled`
→ `configuration.setCacheEnabled(true)`
- `this.environmentsElement(root.evalNode("environments"))`
 - 解析，配置数据源和事务管理器。通常在 Spring 整合时这部分会被 Spring 管理，XML 中可以不配置或忽略。
- `this.databaseIdProviderElement(root.evalNode("databaseIdProvider"))`
 - 解析，根据当前数据库产品提供 `databaseId`，用于多数据库支持
- `this.typeHandlersElement(root.evalNode("typeHandlers"))`
 - 解析，注册自定义的类型处理器。
- `this.mappersElement(root.evalNode("mappers"))`
 - 解析，指定 Mapper 映射文件的位置。支持：

```

<mapper resource="..."/>

<mapper url="..."/>

<mapper class="..."/> (注解方式)

<package name="..."/>

```

◦ 设置环境 (Environment)

```

targetConfiguration.setEnvironment(new Environment(this.environment,
    this.transactionFactory == null ? new
SpringManagedTransactionFactory() : this.transactionFactory,
    this.dataSource));

```

- 为 `Configuration` 设置运行环境，包括事务工厂和数据源。
- 默认事务工厂是 `SpringManagedTransactionFactory`，让 MyBatis 使用 Spring 管理的事务。

◦ 解析 Mapper XML 文件

```

if (this.mapperLocations != null) {
    for (Resource mapperLocation : this.mapperLocations) {
        var xmlMapperBuilder = new
XMLMapperBuilder(mapperLocation.getInputStream(), targetConfiguration,
            mapperLocation.toString(),
targetConfiguration.getSqlFragments());
        xmlMapperBuilder.parse();
    }
}

```

- 点击parse方法，和配置文件一样，mapper文件也是这种解析

```

public void parse() {
    if (!this.configuration.isResourceLoaded(this.resource)) {
        this.configurationElement(this.parser.evalNode("/mapper"));
        this.configuration.addLoadedResource(this.resource);
        this.bindMapperForNamespace();
    }

    this.configuration.parsePendingResultMaps(false);
    this.configuration.parsePendingCacheRefs(false);
    this.configuration.parsePendingStatements(false);
}

```

```

private void configurationElement(XNode context) {
    try {
        String namespace =
context.getStringAttribute("namespace");
        if (namespace != null && !namespace.isEmpty()) {

```

```

        this.builderAssistant.setCurrentNamespace(namespace);
        this.cacheRefElement(context.evalNode("cache-ref"));
        this.cacheElement(context.evalNode("cache"));

        this.parameterMapElement(context.evalNodes("/mapper/parameterMap"));
    };

    this.resultMapElements(context.evalNodes("/mapper/resultMap"));
    this.sqlElement(context.evalNodes("/mapper/sql"));

    this.buildStatementFromContext(context.evalNodes("select|insert|update|delete"));
    } else {
        throw new BuilderException("Mapper's namespace cannot be empty");
    }
    } catch (Exception var3) {
        Exception e = var3;
        throw new BuilderException("Error parsing Mapper XML. The XML location is '" + this.resource + "'. Cause: " + e, e);
    }
}

```

- 遍历所有 `mapperLocations` 指定的资源（例如 `classpath*:mapper/**/*.xml`）。
- 每个 Mapper XML 文件被 `XMLMapperBuilder` 解析，生成 `MappedStatement` 并注册到 `targetConfiguration` 中。
- 我们可以看到他首先读namespace，配置文件中 `namespace="wang.zi.jie.myabtis_study.mapper.UserMapper"`，相当于他拿到了类路径，然后解析 `<select>`、`<insert>` 等 SQL 标签，生成 `MappedStatement`，并注册到 `configuration` 中。
 - 举例，用一个usermapper的文件

```

<parameterMap id="userParamMap" type="com.example.User">
    <parameter property="id" jdbcType="INTEGER" mode="IN"/>
    <parameter property="name" jdbcType="VARCHAR"/>
</parameterMap>

<insert id="insertUser" parameterMap="userParamMap">
    INSERT INTO user (id, name) VALUES (?, ?)
</insert>

```

```

private void parameterMapElement(List<XNode> list) {
    for (XNode parameterMapNode : list) { // 遍历
        每个 <parameterMap>
        String id = parameterMapNode.getStringAttribute("id");
        // 映射的唯一标识
        String type =
            parameterMapNode.getStringAttribute("type"); // 参数对应的 Java 类型（全限定类名或别名）
        Class<?> parameterClass = resolveClass(type); // 将
        type 字符串解析成 class 对象
    }
}

```

```

        List<XNode> parameterNodes =
parameterMapNode.evalNodes("parameter"); // 获取所有 <parameter>
子节点

        List<ParameterMapping> parameterMappings = new
ArrayList<>();

        // 遍历每个 <parameter>
        for (XNode parameterNode : parameterNodes) {
            String property =
parameterNode.getStringAttribute("property"); // Java 对象
的属性名

            String javaType =
parameterNode.getStringAttribute("javaType"); // 属性对应的
Java 类型

            String jdbcType =
parameterNode.getStringAttribute("jdbcType"); // JDBC 类型
(如 VARCHAR, INTEGER)

            String resultMap =
parameterNode.getStringAttribute("resultMap"); // 结果映射 (很
少用于参数)

            String mode =
parameterNode.getStringAttribute("mode"); // 存储过
程参数模式 (IN/OUT/INOUT)

            String typeHandler =
parameterNode.getStringAttribute("typeHandler"); // 自定义类型处理
器

            Integer numericScale =
parameterNode.getIntAttribute("numericScale"); // 数值型参数的保留
小数位

            // 将字符串解析成枚举或 Class
            ParameterMode modeEnum = resolveParameterMode(mode);
            Class<?> javaTypeClass = resolveClass(javaType);
            JdbcType jdbcTypeEnum = resolveJdbcType(jdbcType);
            Class<? extends TypeHandler<?>> typeHandlerClass =
resolveClass(typeHandler);

            // 构建一个 ParameterMapping 对象 (描述单个参数)
            ParameterMapping parameterMapping =
builderAssistant.buildParameterMapping(parameterClass, property,
javaTypeClass, jdbcTypeEnum, resultMap,
modeEnum, typeHandlerClass, numericScale);
            parameterMappings.add(parameterMapping);
        }

        // 将整个参数映射注册到 builderAssistant 中, 最终存入
Configuration
        builderAssistant.addParameterMap(id, parameterClass,
parameterMappings);
    }
}

```

- 此外，在 `configurationElement` 方法解析完一个 Mapper XML 文件后，所有配置信息（`<cache>`、`<resultMap>`、`<sql>`、`<select|insert|update|delete>` 等）都会被存入 **MyBatis 全局配置对象** `Configuration` 中。`Configuration` 内部使用了一系列 `Map` 结构来分类存储，形成类似“注册表”的数据组织方式。

存储内容	数据结构	Key 格式	Value 类型
SQL 语句 (MappedStatement)	Map<String, MappedStatement>	namespace + "." + id (如 com.example.UserMapper.selectById)	MappedStatement (封装 SQL、参数映射、结果映射等)
结果映射 (ResultMap)	Map<String, resultMap>	namespace + "." + id	ResultMap (定义列与 Java 属性的映射关系)
SQL 片段 (SqlSource)	Map<String, xNodes>	namespace + "." + id	xNode (<sql> 片段, 可被 include 引用)
二级缓存 (Cache)	Map<String, Cache>	namespace	Cache (如 PerpetualCache)
缓存引用 (CacheRef)	Map<String, String>	namespace	被引用的其他 namespace
参数映射 (ParameterMap, 已过时)	Map<String, ParameterMap>	namespace + "." + id	ParameterMap (定义参数映射规则)

- 比如上述代码最后一行的 `builderAssistant.addParameterMap(id, parameterClass, parameterMappings)`;点进去调用了 `configuration.addParameterMap(parameterMap)`;

```
public ParameterMap addParameterMap(String id, Class<?> parameterClass, List<ParameterMapping> parameterMappings) {
    id = applyCurrentNamespace(id, false);
    ParameterMap parameterMap = new ParameterMap.Builder(configuration, id, parameterClass, parameterMappings).build();
    configuration.addParameterMap(parameterMap);
    return parameterMap;
}
```

- 再点击 `configuration.addParameterMap(parameterMap)`;，发现进入到 `map` 里了

```
public void addParameterMap(ParameterMap pm) {
    parameterMaps.put(pm.getId(), pm);
}
```

- 我们可以发现他是 `protected final Map<String, MappedStatement> mappedStatements = new StrictMap` 这样类型的，继承了 `extends ConcurrentHashMap<String, V>`，所以 `mybatis` 的 `map` 是 `ConcurrentHashMap`，并发安全的。
- 我们看到解析完执行了 `bindMapperForNamespace` 方法，他是负责利用解析出的 `namespace` 去加载 Java 接口，并将其注册到 `MyBatis` 的 `MapperRegistry` 中，使得接口与刚刚解析的 SQL 映射关联起来。

```
private void bindMapperForNamespace() {
    String namespace = builderAssistant.getCurrentNamespace();
    if (namespace != null) {
        Class<?> boundType = null;
        try {

```



```

        boundType = Resources.classForName(namespace);
    } catch (ClassNotFoundException e) {
        // ignore, bound type is not required
    }
    if (boundType != null && !configuration.hasMapper(boundType))
    {
        // Spring may not know the real resource name so we set a
        flag
        // to prevent loading again this resource from the mapper
        interface
        // look at MapperAnnotationBuilder#loadXmlResource
        configuration.addLoadedResource("namespace:" + namespace);
        configuration.addMapper(boundType);
    }
}
}
}

```

- 关注 `boundType = Resources.classForName(namespace);`, 他使用 MyBatis 的 `Resources` 工具类（封装了类加载器）去加载 `namespace` 对应的类。
- 注册接口到 `MapperRegistry`

```

if (boundType != null && !configuration.hasMapper(boundType)) {
    configuration.addLoadedResource("namespace:" + namespace);
    configuration.addMapper(boundType);
}

```

- `configuration.addLoadedResource(boundType)` 检查这个接口是否已经被注册过了。没有就放进去，用了 `HashSet`
- `configuration.addLoadedResource("namespace:" + namespace)`: 将 `namespace:` 标记为已加载资源，防止后续其他流程（例如从接口注解解析时）重复加载同一个 XML。
- `configuration.addMapper(boundType)` **是关键**:
它内部调用 `mapperRegistry.addMapper(boundType)`，将接口存入 `knownMappers` 这个 `HashMap` 中，并为该接口创建一个 `MapperProxyFactory`。之后当你调用 `getMapper` 时，就会从这个工厂获取动态代理。
- 怎么和之前的sql关联:
 - 在调用 `bindMapperForNamespace()` 之前，`configurationElement()` 已经解析了 XML 中的所有 SQL 标签，并生成了 `MappedStatement`，其 ID 为 `namespace + "." + methodName`。
 - 注册接口后，动态代理在调用方法时，会根据接口全名 + 方法名去 `Configuration.mappedStatements` 中查找对应的 `MappedStatement`，从而执行正确的 SQL。
 - 所以本质上，XML 解析与接口注册是“先有蛋（`MappedStatement`），后有鸡（接口代理）”，但最终通过 `namespace + methodName` 这个共同的 Key 关联起来。